

*****Să se creeze în Kotlin un server TCP care să facă o cerere (khttp) pentru simbolurile companiilor cu acțiuni (vezi documentația: <https://finnhub.io/docs/api#stock-symbols>) și folosind aceste simboluri precum și API-ul de prețuri țintă (<https://finnhub.io/docs/api#price-target>) să trimită datele prin socket câte o companie (symbol) o dată la 3 secunde (pentru serializare/deserializare se pot utiliza funcțiile loads și dumps din modulul json din Python, iar în Kotlin se poate utiliza `kotlinx.serialization` <https://github.com/Kotlin/kotlinx.serialization>). De asemenea, se va implementa în Python un flux de date direct (direct stream) utilizând framework-ul Apache Spark (PySpark), care să preia datele de la serverul TCP și să le prelucereze astfel:

- se va calcula pentru acțiunile fiecărei companii profitul mediu care putea fi făcut în luna curentă, pe baza prețurilor estimate de analiști (`targetMean` și `targetLow`);
- se vor filtra rezultatele, păstrându-le doar pe cele pentru care procentul de profit > 40%
- pentru fiecare RDD în parte, se va afișa compania (symbol) și profitul mediu.

Există o limitare de 60 de apeluri / minut la fiecare cheie API. Se poate crea un cont pe platforma finnhub.io (<https://finnhub.io/register>), sau se poate utiliza următoarea cheie (token) API: `brl7eb7rh5re1lvco7fg`

CEVA GASIT

SERVER KOTLIN, proiect facut in MAVEN cu 2 fisiere -> instructiuni pt creare proiect maven lab 3 SD

POM.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://maven.apache.org/POM/4.0.0"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <artifactId>KotlinServer</artifactId>
  <groupId>org.example</groupId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>KotlinServer</name>
```

```
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <kotlin.code.style>official</kotlin.code.style>
  <kotlin.compiler.jvmTarget>1.8</kotlin.compiler.jvmTarget>
</properties>
```

```
<repositories>
  <repository>
    <id>mavenCentral</id>
    <url>https://repo1.maven.org/maven2</url>
  </repository>
  <repository>
    <id>jcenter</id>
    <url>https://jcenter.bintray.com</url>
  </repository>
</repositories>
```

```
<build>
  <sourceDirectory>src/main/kotlin</sourceDirectory>
  <testSourceDirectory>src/test/kotlin</testSourceDirectory>
  <plugins>
    <plugin>
      <groupId>org.jetbrains.kotlin</groupId>
      <artifactId>kotlin-maven-plugin</artifactId>
      <version>1.6.21</version>
      <executions>
        <execution>
          <id>compile</id>
          <phase>compile</phase>
          <goals>
            <goal>compile</goal>
          </goals>
        </execution>
        <execution>
          <id>test-compile</id>
          <phase>test-compile</phase>
          <goals>
            <goal>test-compile</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
    <plugin>
      <artifactId>maven-surefire-plugin</artifactId>
```

```

        <version>2.22.2</version>
    </plugin>
    <plugin>
        <artifactId>maven-failsafe-plugin</artifactId>
        <version>2.22.2</version>
    </plugin>
</plugins>
</build>

<dependencies>
    <dependency>
        <groupId>khttp</groupId>
        <artifactId>khttp</artifactId>
        <version>1.0.0</version>
    </dependency>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlinx-serialization-json-jvm</artifactId>
        <version>1.3.3</version>
    </dependency>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-test-junit5</artifactId>
        <version>1.6.21</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>5.6.0</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-stdlib-jdk8</artifactId>
        <version>1.6.21</version>
    </dependency>
</dependencies>

</project>

```

FinhubClient.kt

```

import khttp.get
import org.json.JSONObject

class FinnhubClient(private val token: String) {
    private companion object {
        const val base = "https://finnhub.io/api/v1/stock"
    }
    private val symbolsLink = "$base/symbol?exchange=US&token=$token"
    private val priceTagLink = "$base/price-target?token=$token"

    fun getSymbols(): List<String> =
        get(symbolsLink)
            .jsonArray
            .map { JSONObject(it.toString()).getString("symbol") }

    fun getSymbolsData(symbol: String): String = get("$priceTagLink&symbol=$symbol").text
}

```

MAIN.kt

```

import java.lang.Thread.sleep
import java.net.ServerSocket
import java.net.Socket
import kotlin.concurrent.thread

fun main(args: Array<String>) {
    val finnhubClient = FinnhubClient("bri7eb7rh5re1lvco7fg")
    val symbols = finnhubClient.getSymbols()
    val serverSocket = ServerSocket(8080)
    var clientSocket: Socket
    var symbolData: String
    try {
        symbols.forEach {
            clientSocket = serverSocket.accept()
            println("Connected")
            thread {
                symbolData = finnhubClient.getSymbolsData(it)
                clientSocket.getOutputStream().write("$symbolData\n".toByteArray())
                println("Sent data: $symbolData")
                clientSocket.close()
            }
        }
        sleep(1_000)
    }
}

```

```

    }
} catch (ex: Exception) {
    println(ex.toString())
} finally {
    serverSocket.close()
}
}

```

--- PYTHON

```

import os
from json import loads

```

```

from pyspark import SparkContext, SparkConf
from pyspark.streaming import StreamingContext
from functional import seq

```

```

def compute_profit(target_low, target_mean):
    return 40 # Cine stie cum se calcula...

```

```

if __name__ == '__main__':
    # configurare variabila de mediu cu versiunea python utilizata pentru spark
    os.environ['PYSPARK_PYTHON'] = '/usr/bin/python3'
    # configurarea Spark
    spark_conf = SparkConf().setMaster("local[*]").setAppName("Spark Example")
    # initializarea contextului Spark
    spark_context = SparkContext(conf=spark_conf)
    # Spark Streaming Context
    ssc = StreamingContext(spark_context, 3)

    stream = ssc.socketTextStream('localhost', 8080)

    stream.map(lambda it: loads(it)).pprint()

    # stream.map(lambda it: loads(it))\
    #   .filter(lambda it: compute_profit(it['targetLow'], it['targetMean']) >= 40)\
    #   .foreachRDD(lambda rdd: seq(rdd.collect()).for_each(lambda it: print(it))) # Cam sa ar
    #   trebui defapt sa fie

    # RDD-urile din foreachRDD sunt liste. Le transform in secvente cu seq din PyFunctional
    # si pentru fiecare element din secventa afisez...
    stream.map(lambda it: loads(it)).foreachRDD(lambda rdd: seq(rdd.collect()).for_each(lambda
it: print(it)))
    ssc.start()

```

```
ssc.awaitTermination()  
ssc.stop()
```

Pentru testarea exemplului, din IntelliJ se execută Maven -> Lifecycle -> clean + compile + package. La final, se deschide directorul target creat și se execută în terminal comanda : java -jar BeerApp-1.0.0.jar
sau dau run idkk